MANIPAL UNIVERSITY JAIPUR

# RAILWAY RESERVATION SIMULATOR

## with Disruption Refund Advisor

### SOFTWARE REQUIREMENTS SPECIFICATION

Revision 0.2 • 24 October 2026

| Document field | Value |
|---|---|
| Prepared for | CSC2104 - Object-Oriented Programming and CSC2131 - RDBMS Lab |
| Prepared by | Soham More, Department of Computer Science & Engineering |
| Status | Revised draft for review and implementation planning |
| Primary implementation | Standalone Java console simulator and independent MySQL 8 database |
| Optional extension | JDBC / HTTP / frontend only after Tier 1 and Tier 2 acceptance gates |

**Important scope statement**

This is a deterministic educational simulator. It does not connect to live railway inventory, query live PNRs, accept real passenger data, file a TDR, or make an official refund decision. Rule outputs are simulated guidance only.

**Revision v0.2 changes**

The review revision restores the earlier reservation invariants, fixes cross-class booking concurrency, makes filing-attempt time explicit, rejects unverified refund rules as unresolved, normalizes evaluation reasons and evidence links, and removes linked-connection refunds from MVP scope.

# Document control and review status

| Item | Decision / status |
|---|---|
| Previous draft | v0.1 dated 24 October 2026; received as a 16-page PDF |
| This revision | v0.2; changes applied after technical and official-rule review |
| Implementation status | Nothing in this document claims that the software is implemented or tested |
| Blocking prerequisite | Obtain and verify the official CSC2104 course handout before finalizing Java language restrictions |
| Database dialect | MySQL 8.0.16+ with InnoDB; confirm faculty acceptance of MySQL if the lab assessment expects a specific Oracle/PLSQL toolchain |
| Policy source status | Official IRCTC refund rules located; each exact rule and scope must be reviewed by the team before its rule row is marked verified |
| Scope decision | Reservation engine plus two refund-rule families; linked-connection refund and partial-cancellation rules are out of MVP scope |

# Document map

| Section | Contents |
|---|---|
| 1 | Introduction, purpose, scope and definitions |
| 2 | Product overview, users, environment and constraints |
| 3 | Reservation domain rules and invariants |
| 4 | Reservation functional requirements |
| 5 | Disruption refund advisor requirements |
| 6 | Data model, tables, integrity and transaction design |
| 7 | Interfaces and fixture contract |
| 8 | Non-functional requirements |
| 9 | Verification fixtures and acceptance criteria |
| 10 | CSC2104 / CSC2131 syllabus traceability |
| 11 | Implementation phases, ownership and cut order |
| 12 | Risks, assumptions and decisions required |
| 13 | References |
| Appendix A | Canonical reservation and refund fixture matrix |
| Appendix B | Example deterministic case fixture |
| Appendix C | Required RDBMS query / stored-program checklist |

# 1. Introduction

## 1.1 Purpose

This SRS defines a teaching simulator for railway reservations with a small, rule-driven disruption refund advisor. It is the common baseline for implementation, tests, project report and viva. Tier 1 (Java) and Tier 2 (MySQL) must work independently and implement the same domain rules.

## 1.2 Product summary

The reservation engine models trains, dated journeys, seat classes, bookings, unique PNRs, cancellations, waitlists and promotion. It demonstrates why concurrent booking requests can over-allocate a last seat in an unsafe implementation and how a corrected implementation prevents that outcome.

The refund advisor evaluates a disruption case using explicitly supplied event records and an effective-dated, source-referenced rule. It returns an outcome, reason codes, the selected rule/version, deadline information and a simulated refund amount. Java and MySQL evaluate the same deterministic fixtures independently; a parity checker compares their outputs.

## 1.3 In scope

- Reservation: passengers, trains, journeys, journey classes, seat inventory, booking, unique PNR, cancellation, waitlist, promotion and audit.
- Concurrency: Java threads/monitor synchronization and a bounded FIFO queue; MySQL transactions, lock-first booking, controlled two-session experiments and savepoints.
- Refund advisor: two rule families only - train running more than three hours late with passenger not travelling, and train cancelled - scoped to confirmed simulated e-tickets.
- Rules as data: effective-dated versions, official source reference, review date, verification flag and explicit applicability time.
- Deterministic case events, outcome/reason codes, normalized evaluation-code records, evidence-event links and plain-text case summary.
- Shared fixtures and parity tests that compare outcome, reason-code set, deadline/action, warning set and refund amount.

## 1.4 Out of scope

- Live train position/status, live PNR enquiry, real booking inventory, real payments or IRCTC/RailMadad API integration.
- Collecting IRCTC passwords, real PNRs or real passenger personal data; filing a real TDR or grievance.
- RAC, waitlisted, partially confirmed, party/family PNR, Tatkal and i-ticket refund adjudication. These may exist in reservation simulation only where explicitly represented, but the advisor refuses unsupported ticket types.
- Connecting-train / missed-connection refund advice. The current IRCTC-train-specific rule page states that connecting-train booking is not allowed on IRCTC trains and no refund applies on connecting-journey tickets. Any broader case requires separate official scope validation before inclusion.
- Partial-cancellation rules, AC failure, coach-not-attached, diverted/short-terminated trains, machine learning and production web hosting.
- Legal, financial or official refund advice. Every case result is explicitly labelled a simulation.

## 1.5 Definitions

| Term | Definition |
|---|---|
| PNR | Unique simulated booking reference. A PNR is never reused. |

| Term | Definition |
|---|---|
| Journey | One train service on a specific date for a defined boarding-to-destination segment. |
| Journey class | One seat class for one journey, such as Sleeper, AC or General. |
| Group reference | Identifier shared by multiple individual bookings; each booking still has its own PNR. |
| Active allocation | Current seat allocation for a CONFIRMED booking. |
| Waitlist entry | Queue record for a WAITLISTED booking, ordered by booking time and then booking ID. |
| Disruption case | A record linked to one eligible simulated booking and a disruption reason. |
| Case event | A timestamped fact with event time, recorded time, type and source label. |
| Rule version | A dated, source-referenced refund policy row with a version and verification state. |
| Evaluation | Immutable result computed from a fixed rule version, explicit event set and evaluation time. |
| TDR | Ticket Deposit Receipt; a refund-claim process for certain cases. The simulator does not file it. |
| Parity | Java and MySQL produce the same normalized outcome, reason-code set, deadline/action, warnings and amount for a fixture. |

# 2. Product overview and operating constraints

## 2.1 Architecture tiers

| Tier | Component | Required independence |
|---|---|---|
| Tier 1 | Standalone Java console simulator, domain model, booking agents, queue, safe/unsafe concurrency experiments, refund-rule classes and file audit. | Runs without MySQL, JDBC or network. |
| Tier 2 | Independent MySQL 8 schema, constraints, views, queries, functions, procedures, triggers, cursor, audit, transactions and two-session experiments. | Runs without Java. |
| Tier 3 (optional) | JDBC adapter, localhost HTTP API and small frontend. | May begin only after Tier 1 and Tier 2 acceptance gates pass; must not delay the assessed cores. |

## 2.2 Users

| User | Main use |
|---|---|
| Booking clerk (simulated) | Create passenger, book/cancel, look up PNR, view availability and waitlist. |

| User | Main use |
|---|---|
| Simulated passenger | Open a disruption case, add timestamped facts, evaluate it and read the rule-based case summary. |
| Evaluator | Run Java scenarios, SQL scripts, two-session race experiments, fixtures and parity checks; conduct viva. |
| Maintainer / rule reviewer | Add a new effective-dated rule through a controlled procedure and record its official source/review metadata. |

### 2.3 Operating environment

- Java: course-approved JDK; console execution; file system for audit logs and fixtures.
- Database: MySQL 8.0.16 or later, InnoDB, MySQL Workbench or CLI. MySQL syntax only; where a lab topic is taught using Oracle/PLSQL terminology, document the MySQL equivalent and limitations.
- No network is required for Tier 1 or Tier 2. Official policy research is an implementation-time activity; the running application uses seeded rule data only.
- All journey timestamps are interpreted in Asia/Kolkata (IST) for this simulator. Fixture inputs must include explicit timestamps and must not read the machine clock during evaluation.

### 2.4 Java course restrictions (provisional)

The earlier project plan assumes a fixed-array bounded queue and monitor-based synchronization using synchronized, wait and notifyAll, and excludes java.util.concurrent and the Collections Framework from the assessed core if the official CSC2104 handout requires this. This is not yet verified. The Java queue must not be coded until the official handout is checked; if its rules differ, update the implementation constraints while preserving the concurrency acceptance tests.

### 2.5 Assumptions

- Demo-scale dataset: a few trains, journeys and tens of seats per class; all data is synthetic.
- One booking reserves one passenger/seat. A group reservation is several individual bookings with a shared group reference, not one multi-passenger PNR.
- The refund advisor initially accepts only CONFIRMED bookings with ticket type SIMULATED_E_TICKET. Refund facts are simulated; no real status is fetched.
- Only rules whose exact scope and source have been reviewed may be marked verified. A database row with verified = false cannot produce ELIGIBLE or NOT_ELIGIBLE.
- The team must confirm the actual course submission deadline and whether MySQL is accepted for the assessed RDBMS deliverable.

## 3. Reservation domain rules and invariants

### 3.1 Booking lifecycle

| Current status | Allowed next status | Notes |
|---|---|---|
| PENDING | CONFIRMED, WAITLISTED, CANCELLED | Initial booking state; status change is audited. |
| CONFIRMED | CANCELLED | Cancellation releases exactly one allocation and attempts one eligible promotion. |
| WAITLISTED | CONFIRMED, CANCELLED | Promotion only; cancellation releases no seat. |
| CANCELLED | None | Terminal; never reactivated. A new request receives a new booking/PNR. |

## 3.2 Business rules

- BR-01: PNR is unique and never reused.
- BR-02: one booking holds one seat. A group reference can join multiple independent bookings.
- BR-03: a passenger may have at most one CONFIRMED booking per journey, across all seat classes. WAITLISTED bookings in other classes may remain as fallbacks.
- BR-04: waitlist order is earliest booking timestamp first, then booking ID as a deterministic tie-break.
- BR-05: when no seat is available, a valid request becomes WAITLISTED; this is a normal outcome, not an exception.
- BR-06: a CANCELLED booking is terminal. Re-booking creates a new booking and PNR.
- BR-07: cancelling a CONFIRMED booking releases exactly one seat and makes at most one attempt to promote the earliest eligible waiter in that journey class. Cancelling a WAITLISTED booking frees no seat.
- BR-08: fares are non-negative and depend on seat class.
- BR-09: promotion skips a passenger who already has a confirmed booking on the same journey in another class; it must not mutate an ineligible waitlist entry.
- BR-10: booking status and seat/waitlist/allocation tables are changed only through controlled Java service operations or the approved MySQL procedures.

## 3.3 Invariants

| ID | Invariant | Verification |
|---|---|---|
| INV-01 | Confirmed bookings never exceed the number of seats in the matching journey class. | Count CONFIRMED bookings/allocations by journey class. |
| INV-02 | A seat has at most one active allocation and a booking has at most one active allocation. | Unique constraints plus query checks. |
| INV-03 | Each CONFIRMED booking has exactly one allocation; other statuses have none. | Left-join verification query returns zero violations. |
| INV-04 | Each WAITLISTED booking has exactly one waitlist entry; CANCELLED/CONFIRMED bookings have none. | Left-join verification query returns zero violations. |
| INV-05 | A passenger has at most one CONFIRMED booking per journey across all classes. | Journey-level serialization plus invariant query. |
| INV-06 | Every committed booking creation/status transition has matching audit history; rolled-back work leaves no committed audit row. | Compare booking lifecycle with BOOKING_AUDIT. |
| INV-07 | Every waitlist promotion chooses the earliest eligible row under BR-04/BR-09 and promotes at most one booking per cancellation. | Deterministic waitlist fixture. |

## 3.4 Required lock order

**Critical correction from v0.1**

Locking only JOURNEY_CLASS cannot protect BR-03, because two concurrent requests can lock different classes for the same journey. A common JOURNEY row is therefore the first serialization/lock anchor; the relevant JOURNEY_CLASS row is locked second. All booking, cancellation and promotion operations use the same documented order.

1. Begin a MySQL transaction. The first data-access statement must be a locking read (SELECT ... FOR UPDATE) of the JOURNEY parent row for the target journey. No plain availability/count read may occur first.
2. Lock the affected JOURNEY_CLASS row next. When multiple rows are needed, lock them in deterministic key order; do not acquire a class lock before the parent journey lock.
3. While these locks are held, validate immutable booking route/class identifiers, status, passenger journey-level confirmed status and available seats; then insert/change booking, allocation or waitlist rows.
4. Write audit rows in the same transaction and commit. On any error, roll back the whole operation. Deadlocks/lock timeouts are recorded as failed attempts and may be retried by the caller; the retry must not duplicate the logical booking.

For cancellation, the procedure receives booking ID plus expected immutable journey/class identifiers (or an equivalent safe lookup form). It locks JOURNEY, then JOURNEY_CLASS, then validates that the booking really belongs to those identifiers. A mismatched identifier is rejected without mutation. The pre-lock lookup, if used by a caller to discover immutable identifiers, is not used to determine availability or make a capacity decision.

# 4. Functional requirements: reservation core

Priority legend: M = must be complete for the core submission; S = should be completed if time permits after M requirements; C = could be attempted only after acceptance gates pass.

| ID | Requirement | Priority |
|---|---|---|
| **FR-R01** | Model TRAIN, JOURNEY, JOURNEY_CLASS and fixed JOURNEY_SEAT inventory. Include at least two trains and all three simulated classes: Sleeper, AC and General. Include a zero-seat and a one-seat class in fixtures. | M |
| **FR-R02** | Create a PENDING booking request with generated unique PNR; never reuse a PNR, including after cancellation or rollback of a committed booking. | M |
| **FR-R03** | One booking allocates one seat. Multiple members of a group are separate bookings with individual PNRs and a shared group reference. | M |
| **FR-R04** | Confirm when eligible inventory exists; otherwise set WAITLISTED. Full capacity is a normal outcome. | M |
| **FR-R05** | Enforce at most one CONFIRMED booking per passenger per journey across all classes. | M |
| **FR-R06** | List waitlist in deterministic booked_at, booking_id order. | M |
| **FR-R07** | Cancel confirmed booking: remove one current allocation, set CANCELLED and attempt one earliest eligible promotion. Cancel waitlisted booking: remove waitlist entry and set CANCELLED without releasing a seat. | M |
| **FR-R08** | Enforce the lifecycle in section 3.1. Illegal transitions raise a domain error/SQL error and leave no partial committed changes. | M |
| **FR-R09** | Calculate non-negative fare by class using | M |

| ID | Requirement | Priority |
|---|---|---|
| | an abstract SeatClass type and overriding class policy in Java. | |
| **FR-R10** | Console supports show availability, create/list passenger, book, cancel by PNR, look up PNR, show waitlist, run race demo, open refund case, add case event, evaluate case, print summary and exit. Invalid input returns to the menu. | M |
| **FR-R11** | Audit every booking creation and status transition. Java uses a synchronized file logger; MySQL uses transactional audit rows inserted by triggers. | M |
| **FR-R12** | Implement a BookingBackend interface; console and booking agents depend on the interface. Java in-memory service is one implementation. | M |
| **FR-R13** | Implement a bounded FIFO queue using the fixed-array/monitor approach required by the verified course handout. Queue shutdown rejects new submissions, drains queued requests, wakes waiters and joins all workers. | M |
| **FR-R14** | Run at least three booking-agent threads. Correctness must not depend on thread priority or favourable timing. Use Thread/Runnable as permitted by the handout. | M |
| **FR-R15** | Keep a separate unsafe check-then-claim experiment with a staged start gate that makes two workers observe the same last seat before claiming. Show the violation in a test-only inventory. | M |
| **FR-R16** | Run the corrected Java inventory against the same contention workload with 10 workers over at least 100 repetitions; capacity must never be exceeded. | M |
| **FR-R17** | Cancellation and promotion obey the common lock order and BR-09. An ineligible first waitlisted booking is skipped without mutation; the next eligible row may be promoted. | M |
| **FR-R18** | Group booking uses one transaction and a savepoint for each of at least three member reservations. A deliberately failing member rolls back only work since its savepoint; other valid member bookings commit once. | M |
| **FR-R19** | Provide a test checklist with fixture ID, setup, expected result, observed result and evidence. | M |

# 5. Functional requirements: disruption refund advisor

## 5.1 Initial supported cases

| Rule family | MVP behavior | Source / conditions |
|---|---|---|
| LATE_OVER_3_HOURS | Full-fare eligibility only when train delay is strictly >180 minutes at the modeled boarding/commencement station, the passenger did not travel, and TDR filing attempt occurred before actual departure. | IRCTC Refund Rules, clause 6; current official PDF. The simulator models a single confirmed e-ticket. |
| TRAIN_CANCELLED | For a confirmed simulated e-ticket, model automatic full-fare refund; no cancellation/TDR action is required in this rule path. | IRCTC Refund Rules, clause 5(a), which says confirmed/RAC e-tickets receive automatic full refund; this project models confirmed only. |

**Removed from v0.2 MVP: connecting-train refund**

The IRCTC-train-specific official rules page states that connecting-train ticket booking is not allowed on IRCTC trains and that no refund applies on connecting-journey tickets. The prior linked-connection rule has therefore been removed from this specification, not merely marked as a stretch goal. Reconsider only if a distinct, applicable official rule and ticket context are verified.

## 5.2 Scope and rule-source requirements

- The advisor accepts a case only for a CONFIRMED booking with ticket_type = SIMULATED_E_TICKET. Waitlisted, RAC, partial-confirmed, party/family PNR, Tatkal and i-ticket cases are unsupported and must be rejected with an explicit unsupported-scope reason, not guessed.
- No live train-status or ticket data is read. Case events are simulated records and use source labels such as SIMULATED_OFFICIAL and PASSENGER_REPORTED. SIMULATED_OFFICIAL denotes a trusted source in a test fixture; it does not mean that the system queried Indian Railways.
- Every REFUND_RULE version contains official source URL, source title/section, source_checked_on, verification status, ticket scope, threshold, deadline basis and refund basis. A rule cannot be marked verified unless its source details and review date are present.
- An unverified rule must return UNRESOLVED with RULE_SOURCE_UNVERIFIED; it cannot produce ELIGIBLE or NOT_ELIGIBLE. Test fixtures may explicitly mark a source-reviewed synthetic rule as verified for deterministic evaluation.
- Effective intervals use [effective_from, effective_to); effective_to may be NULL for an open interval. Overlapping intervals for the same rule family are rejected by a controlled rule-maintenance procedure.
- Rule applicability time and evaluation time are different fields. The fixture supplies policy_applicability_time explicitly; for the initial rule set it defaults to the modeled scheduled departure at the boarding/commencement station. This is a documented implementation assumption and must be checked against the source when publishing each rule. Evaluation never uses the system wall clock.
- Case output contains the selected rule ID/version, outcome, normalized reason-code set, warnings, deadline timestamp or deadline basis, simulated refund amount, required filing action (if any), and the mandatory disclaimer.

## 5.3 Event types and source labels

| Event type | Required value / use |
|---|---|
| SCHEDULED_DEPARTURE | Scheduled departure timestamp at the modeled boarding/commencement station. |
| DELAY_AT_BOARDING_MINUTES | Non-negative integer delay minutes observed at the boarding/commencement station; required source class: SIMULATED_OFFICIAL. |

| Event type | Required value / use |
|---|---|
| ACTUAL_DEPARTURE | Actual departure timestamp used as the deadline cutoff for the late-running rule; source class SIMULATED_OFFICIAL. |
| TRAIN_CANCELLED | Cancellation fact for the modeled journey; source class SIMULATED_OFFICIAL. |
| PASSENGER_TRAVELLED | Explicit fact that the passenger travelled; source label recorded. |
| PASSENGER_NOT_TRAVELLED | Explicit fact that the passenger did not travel; absence of PASSENGER_TRAVELLED alone is not proof of non-travel. |
| FILING_ATTEMPT | Explicit simulated TDR filing-attempt timestamp. Required to adjudicate whether a late-running claim was filed before actual departure. |

Every CASE_EVENT stores event_time (when the fact applies), recorded_at (when it was entered), source_label, and any type-specific value. Duplicate facts that conflict are not silently resolved: the evaluation reports the conflicting/missing fact as unresolved.

## 5.4 Outcome semantics and precedence

| Outcome | Exact condition |
|---|---|
| UNRESOLVED | No applicable rule; rule source unverified; required fact absent; source/time/scope conflict; unsupported ticket type; unknown travel state; or filing attempt missing when needed. Use RULE_NOT_FOUND, RULE_SOURCE_UNVERIFIED, DELAY_UNKNOWN, DELAY_UNVERIFIED, TRAVEL_STATUS_UNKNOWN, FILING_ATTEMPT_UNKNOWN or another precise reason as applicable. |
| NOT_ELIGIBLE | A verified rule and complete trusted facts show an eligibility condition is not met, e.g. delay is <=180 minutes or passenger travelled. |
| DEADLINE_EXPIRED | The other late-running eligibility conditions are satisfied, but an explicit FILING_ATTEMPT time is equal to or after the actual-departure cutoff. Do not infer this outcome from evaluation time alone. |
| ELIGIBLE | A verified rule and complete facts meet all required conditions. For train cancellation in this model, outcome is ELIGIBLE with TRAIN_CANCELLED_AUTO_REFUND and action NO_TDR_REQUIRED. For late running, filing attempt must be strictly earlier than actual departure. |

## 5.5 Evaluation order

5. Validate booking exists, is CONFIRMED, is in the supported simulated e-ticket scope, and case reason type matches its rule family.
6. Select the applicable rule by the explicit policy_applicability_time and effective interval; if none exists, return UNRESOLVED / RULE_NOT_FOUND.
7. If rule source is unverified, return UNRESOLVED / RULE_SOURCE_UNVERIFIED regardless of numeric conditions.
8. Check all rule-required events, provenance, event-time consistency and conflicting facts. Missing/untrusted facts produce UNRESOLVED, not NOT_ELIGIBLE.
9. For cancellation, require a SIMULATED_OFFICIAL TRAIN_CANCELLED event and confirmed simulated e-ticket; return ELIGIBLE, full modelled fare and NO_TDR_REQUIRED.
10. For late running, require official delay minutes, actual departure, explicit not-travelled fact and filing attempt. If passenger travelled, return NOT_ELIGIBLE. If delay <=180, return NOT_ELIGIBLE. If filing

attempt >= actual departure, return DEADLINE_EXPIRED. If filing attempt < actual departure, return ELIGIBLE for the modeled full fare.

11. Persist the evaluation snapshot, exact event IDs used, rule version, codes, warning set, amount, deadline/action and deterministic text summary atomically.

### 5.6 Reason codes

| Area | Canonical codes (MVP) |
|---|---|
| Reservation | ILLEGAL_TRANSITION, CAPACITY_FULL_WAITLISTED, NO_ELIGIBLE_WAITER, PASSENGER_ALREADY_CONFIRMED, ALLOCATION_MISMATCH. |
| Rule/source | RULE_NOT_FOUND, RULE_SOURCE_UNVERIFIED, RULE_INTERVAL_CONFLICT, UNSUPPORTED_TICKET_SCOPE. |
| Delay facts | DELAY_OVER_THRESHOLD, DELAY_NOT_OVER_THRESHOLD, DELAY_UNKNOWN, DELAY_UNVERIFIED, ACTUAL_DEPARTURE_UNKNOWN. |
| Travel/deadline | PASSENGER_TRAVELLED, TRAVEL_STATUS_UNKNOWN, FILING_ATTEMPT_UNKNOWN, FILED_AFTER_DEPARTURE, FILED_BEFORE_DEPARTURE. |
| Cancellation | TRAIN_CANCELLED_AUTO_REFUND, TRAIN_CANCELLATION_UNKNOWN. |
| Evaluation/parity | EVENT_CONFLICT, EVENT_SCOPE_MISMATCH, EVALUATION_FAILED, PARITY_MATCH, PARITY_MISMATCH. |

Each outcome is accompanied by zero or more normalized reason codes and warnings. Ordering in the display is deterministic: primary outcome reason first, then supporting facts in canonical code order.

### 5.7 Mandatory disclaimer

**Exact output text**

“Simulated guidance based on the rule version shown. This is not an official refund decision. This system does not file a TDR or guarantee a refund.”

## 6. Data model and integrity requirements

### 6.1 Reservation tables (9)

| Table | Purpose / key constraints |
|---|---|
| PASSENGER | Synthetic passenger identity; primary key passenger_id. No real personal data. |
| TRAIN | Train number/name for the simulator; unique train number. |
| JOURNEY | Train + service date + modeled segment/stations + scheduled departure. Common journey-level lock anchor. Unique key prevents duplicate identical journey definition. |
| JOURNEY_CLASS | One row per journey/class and base fare; UNIQUE(journey_id, class_code). Secondary lock anchor. |
| JOURNEY_SEAT | One row per physical simulated seat; UNIQUE(journey_class_id, seat_number). Capacity is derived by counting these rows. |
| BOOKING | PNR (unique and never reused), passenger, journey class, fare DECIMAL(10,2), status, booked_at, optional group_ref and ticket_type. |

| Table | Purpose / key constraints |
|---|---|
| SEAT_ALLOCATION | Current active assignment; UNIQUE(booking_id) and UNIQUE(seat_id). Remove on cancellation; booking/audit history remains. |
| WAITLIST_ENTRY | One queue entry per WAITLISTED booking; UNIQUE(booking_id); deterministic order by created_at then booking_id. |
| BOOKING_AUDIT | Append-only lifecycle/status audit with timestamp, actor, PNR/booking, class, event type, old/new status. |

## 6.2 Refund-advisor tables (7)

| Table | Purpose / key constraints |
|---|---|
| REFUND_RULE_FAMILY | One stable lock/control row per rule reason type; used to serialize rule-version publication and overlap checks. |
| REFUND_RULE | Effective-dated version, applicability basis, thresholds, deadline/refund basis, ticket scope, source URL/title/clause, source_checked_on, reviewer and verified flag. UNIQUE(reason_type, version). |
| DISRUPTION_CASE | Case linked to an eligible booking and one reason type; stores opened_at and case status. |
| CASE_EVENT | Event type, event_time, recorded_at, source label and type-specific value. Parent case FK. |
| CASE_EVALUATION | Immutable result snapshot: case, selected rule/version, policy_applicability_time, evaluation_time, outcome, deadline/action basis and amount. |
| CASE_EVALUATION_CODE | One row per outcome reason/warning code; key includes evaluation_id, code and code_kind (REASON/WARNING). No delimited code list. |
| CASE_EVALUATION_EVENT | Join table linking an evaluation to every CASE_EVENT it used, including the event role in the evaluation. Enables reproducibility after new events are added. |

### Normalization requirement

The schema has 16 tables. Document each table's primary/candidate keys, functional dependencies and 3NF justification. Do not store reason codes as comma-separated text or duplicate capacity totals; derive class capacity from JOURNEY_SEAT rows.

## 6.3 Common data rules

- Use InnoDB. All monetary columns use DECIMAL(10,2) in MySQL and BigDecimal in Java; binary floating-point types are not used for fares/refunds.
- Use explicit timestamps. MySQL DATETIME values in the simulator are interpreted as Asia/Kolkata local time; fixture timestamps are deterministic and include offsets or an explicit IST convention in fixture documentation.
- Use PRIMARY KEY, FOREIGN KEY, UNIQUE, NOT NULL and enforced CHECK constraints wherever applicable. MySQL 8.0.16+ is required for enforced CHECK constraints.
- No direct account access should allow arbitrary changes to BOOKING.status, SEAT_ALLOCATION or WAITLIST_ENTRY; state changes go through stored procedures/approved service methods.

- No overlapping effective intervals for the same rule family. Rule publication locks REFUND_RULE_FAMILY, validates ranges, inserts a new version and never silently edits an already-used published version.
- CASE_EVALUATION and its code/event links are written in one transaction. Later events create a new evaluation; they do not rewrite the historical snapshot.
- An unverified seed rule remains disabled for final simulated adjudication until source and scope review is recorded.

## 6.4 Stored procedures, functions and triggers

| Routine / mechanism | Required responsibility |
|---|---|
| book_seat | Transaction; first read locks JOURNEY row FOR UPDATE; lock JOURNEY_CLASS second; validate passenger journey status; insert PENDING, allocate or waitlist, transition, audit, commit; rollback on error. |
| cancel_booking | Same lock order; validate booking belongs to provided journey/class; cancel, remove allocation/waitlist entry, make at most one eligible promotion attempt, audit and commit. |
| promote_waitlist | Cursor visits waitlist by booked_at and booking_id; skips a passenger already confirmed on the journey; promotes the first eligible booking only; does not commit its caller's transaction. |
| book_group | One transaction, shared group_ref, at least three member requests with savepoints. A scoped failed member rolls back to its savepoint; valid members commit once. |
| occupancy_pct | Return percentage 0.00 to 100.00 based only on current confirmed allocations; zero-seat class returns 0.00; use COALESCE and document READS SQL DATA semantics. |
| evaluate_case | Select applicable verified rule by explicit applicability time; validate facts; calculate outcome/codes/action/amount; persist immutable snapshot and event/code links atomically. |
| Triggers | BEFORE INSERT ensures initial booking status PENDING; BEFORE UPDATE rejects illegal transitions with SIGNAL SQLSTATE 45000; AFTER INSERT and relevant AFTER UPDATE write audit rows. Triggers never COMMIT/ROLLBACK. |
| refresh/report routine | Use a cursor where useful to refresh a section/class summary and provide a concrete course demonstration; cursor declarations/handlers follow MySQL syntax/order. |

## 6.5 Transactional concurrency details

- All status changes and corresponding audit entries commit or roll back together.
- Lock hierarchy is JOURNEY -> JOURNEY_CLASS -> booking/allocation/waitlist rows in deterministic ascending key order. Code must not acquire these in reverse order.
- Booking and cancellation procedures must not read available-seat counts before obtaining the common JOURNEY lock.
- No trigger commits or rolls back. Transaction ownership belongs to the top-level procedure; helper/promotion procedures do not commit.
- An unsafe race experiment uses isolated demo tables without protective uniqueness/locking; it must never corrupt the valid reservation schema.
- The corrected two-session experiment shows session 2 waiting on the JOURNEY row, then observing the committed result and waitlisting or failing safely.

# 7. Interfaces and fixture contract

## 7.1 Java console interface

| Menu action | Expected behavior |
|---|---|
| Show availability | Display journey/class, total seats, confirmed allocations, remaining capacity and occupancy. |
| Passenger management | Create/list synthetic passengers; validate identifiers and input. |
| Book / group book | Return PNR, status and fare; clearly state CONFIRMED or WAITLISTED. |
| Cancel / lookup PNR | Validate booking lifecycle and print current status/history. |
| Show waitlist | List only WAITLISTED bookings in deterministic queue order. |
| Run concurrency demo | Run isolated unsafe/safe experiments and print claims versus capacity and run count. |
| Refund case actions | Open case for supported confirmed simulated e-ticket, add explicit events, evaluate, show plain-text summary. |
| Exit | Stop intake, drain queued requests, wake waiters and join workers cleanly. |

## 7.2 MySQL interface and script order

Provide SQL scripts with a predictable order:

12. 00_reset_schema.sql - targets only the dedicated disposable simulator schema and is safe to rerun.
13. 01_create_tables.sql - creates 16 InnoDB tables, keys, checks and indexes.
14. 02_seed_data.sql - inserts deterministic passengers, journeys/classes/seats, bookings and the source-reviewed seed-rule fixtures.
15. 03_queries_views.sql - required lab SELECTs, joins, aggregates, subqueries and views.
16. 04_functions.sql - occupancy_pct and any required refund calculation helper.
17. 05_triggers.sql - status validation and booking audit.
18. 06_procedures.sql - booking, cancellation, promotion, group savepoint, rule publication and case evaluation.
19. 07_experiments.sql plus a separate two-session run guide - unsafe race, corrected locking, stale-snapshot comparison.
20. 08_verify_invariants.sql - returns zero rows for each invariant on valid data and demonstrably finds the seeded unsafe fault.
21. 09_privileges.sql - least-privilege GRANT/REVOKE demo; optional only if lab permissions prohibit account creation.

## 7.3 Shared fixture schema

Each fixture defines the same logical inputs for Java and MySQL: fixture ID/version, booking/journey state if relevant, rule row/version, event rows with source/time/scope, explicit policy-applicability time, explicit evaluation time, expected outcome, ordered reason-code set, warning set, deadline/action and amount. Both engines must produce machine-readable output with the same field meanings.

## 7.4 Example output

```
Outcome: ELIGIBLE
Reason: DELAY_OVER_THRESHOLD
Rule: LATE_OVER_3_HOURS_E_TICKET / version 1
Delay at modeled boarding station: 210 minutes
```

Passenger travelled: No (explicit simulated fact)
Filing attempt: 14:20 IST; actual departure cutoff: 14:35 IST
Deadline result: FILED_BEFORE_DEPARTURE
Simulated refund amount: INR 1,250.00
Action: This scenario models a full-fare TDR claim filed before actual departure.
Disclaimer: Simulated guidance based on the rule version shown. This is not an official refund decision. This system does not file a TDR or guarantee a refund.

Example only. Values are synthetic and are not a statement about any real passenger or refund entitlement.

# 8. Non-functional requirements

| ID | Requirement | Acceptance evidence |
|---|---|---|
| NFR-01 | Determinism: same fixture, rule version and explicit evaluation time produce the same normalized result. | Repeat fixture execution; compare canonical JSON/result rows. |
| NFR-02 | Independence: Tier 1 runs without database/network; Tier 2 without Java. | Build and run each from its README. |
| NFR-03 | Correctness: all defined reservation invariants hold after each committed operation. | Verification SQL and Java assertions return zero violations. |
| NFR-04 | Concurrency safety: corrected booking operations never over-allocate in repeated staged contention tests. | At least 100 safe-run repetitions with 10 workers; record seed and result. |
| NFR-05 | Transactional integrity: failed operation leaves no partial booking, allocation, waitlist or audit state. | Rollback/savepoint fixtures and audit queries. |
| NFR-06 | Explainability: each refund result identifies rule/version, events used, outcome, reason codes, warnings, deadline/action and amount. | Case summary and CASE_EVALUATION_EVENT links. |
| NFR-07 | Honesty: unverified rules and missing facts remain unresolved; every output contains the mandatory disclaimer. | Negative fixtures and output assertions. |
| NFR-08 | Privacy: only synthetic passengers/bookings; no passwords, real PNRs or real payment details. | Repository and fixture audit. |
| NFR-09 | Reproducibility: a second person can reset, seed and run all core experiments from README instructions. | Fresh checkout walkthrough. |
| NFR-10 | Security: least privilege for the database engine; no routine should be granted global privileges unnecessarily. | GRANT/REVOKE and SHOW GRANTS checks where supported. |
| NFR-11 | Scope: demo-scale data; optional Tier 3 cannot delay Tier 1/2 gates. | Acceptance-gate checklist. |

# 9. Verification and acceptance tests

## 9.1 Reservation fixtures

| ID | Scenario | Expected result |
|---|---|---|
| R-01 | Two Java agents compete for last seat in deliberately unsafe implementation with staged start gate. | Unsafe test inventory records over-claim, proving the race is reproducible. |
| R-02 | Same workload through corrected Java service; 10 workers over 100 repetitions. | No run exceeds capacity; at most one confirmed booking for the last seat. |
| R-03 | Two MySQL sessions book last seat using lock-first procedure. | Session 2 waits for JOURNEY lock, then sees correct current state and cannot overbook. |
| R-04 | Cancel confirmed booking with two waitlisted bookings. | One allocation released; earliest eligible waiter is promoted once. |
| R-05 | First waitlisted passenger is already confirmed in another class of same journey. | First candidate is skipped without mutation; next eligible candidate is promoted. |
| R-06 | Cancel waitlisted booking / cancel with empty waitlist. | No seat released for waitlisted cancellation; empty list is handled cleanly. |
| R-07 | Attempt illegal transition, such as CANCELLED -> CONFIRMED. | Rejected with ILLEGAL_TRANSITION; state and audit remain unchanged. |
| R-08 | Group reservation of three requests; second member deliberately fails after partial insert. | Rollback to that member's savepoint removes its partial work; other valid members commit once. |
| R-09 | Any failed booking operation causes rollback. | No allocation, waitlist, status or audit record from failed transaction remains committed. |
| R-10 | Run all INV-01 to INV-07 checks on valid seed data. | Zero violations. A separate unsafe-only fixture must cause at least one checker to report the intended violation. |

## 9.2 Refund-advisor fixtures

| ID | Input condition | Expected outcome / code |
|---|---|---|
| D-01 | Verified rule; official simulated delay 210 minutes; not travelled; filing 15 minutes before actual departure. | ELIGIBLE / DELAY_OVER_THRESHOLD + FILED_BEFORE_DEPARTURE; full modelled fare. |
| D-02 | Delay exactly 180 minutes; all facts complete and verified. | NOT_ELIGIBLE / DELAY_NOT_OVER_THRESHOLD. |
| D-03 | Delay 179 minutes. | NOT_ELIGIBLE / DELAY_NOT_OVER_THRESHOLD. |
| D-04 | Delay 181 minutes but only passenger-reported delay, no simulated-official delay evidence. | UNRESOLVED / DELAY_UNVERIFIED. |
| D-05 | No delay fact or no explicit travel-status fact. | UNRESOLVED / DELAY_UNKNOWN or TRAVEL_STATUS_UNKNOWN. |
| D-06a | Otherwise eligible; filing attempt one second before actual departure. | ELIGIBLE / FILED_BEFORE_DEPARTURE. |
| D-06b | Otherwise eligible; filing attempt exactly at actual departure. | DEADLINE_EXPIRED / FILED_AFTER_DEPARTURE. |

| ID | Input condition | Expected outcome / code |
|---|---|---|
| D-06c | Otherwise eligible; filing attempt one second after actual departure. | DEADLINE_EXPIRED / FILED_AFTER_DEPARTURE. |
| D-06d | No FILING_ATTEMPT event. | UNRESOLVED / FILING_ATTEMPT_UNKNOWN; evaluation time alone does not prove late filing. |
| D-07 | Official simulated passenger-travelled fact. | NOT_ELIGIBLE / PASSENGER_TRAVELLED. |
| D-08 | Verified official simulated train-cancelled event, confirmed simulated e-ticket. | ELIGIBLE / TRAIN_CANCELLED_AUTO_REFUND; full modelled fare; NO_TDR_REQUIRED. |
| D-09 | Two rule versions; policy-applicability time falls in only one version's [from,to) interval; evaluation occurs later. | Select interval-applicable version, not simply latest inserted row. |
| D-10 | No rule effective at policy-applicability time. | UNRESOLVED / RULE_NOT_FOUND. |
| D-11 | Matching rule has verified=false. | UNRESOLVED / RULE_SOURCE_UNVERIFIED; no definitive eligibility outcome. |
| D-12 | Booking is WAITLISTED or unsupported ticket type. | Reject advisor evaluation with UNSUPPORTED_TICKET_SCOPE; do not infer refund eligibility. |
| D-13 | Same fixture evaluated at fixed explicit time in Java and MySQL. | Exact outcome, reason-code set, warnings, rule ID/version, deadline/action and amount match. |
| D-14 | Conflicting official-simulated facts for same essential event. | UNRESOLVED / EVENT_CONFLICT. |

### 9.3 Parity contract

The parity runner compares canonical values, not human prose alone. Required fields: outcome; sorted reason-code set; sorted warning set; selected rule ID/version; policy-applicability timestamp; deadline timestamp/basis; action-required code; currency; and amount to two decimals. When they disagree, emit PARITY_MISMATCH with field-level difference. A parity pass does not prove the policy itself is correct; it proves both implementations agree with the fixture oracle.

## 10. Course traceability

### 10.1 CSC2104 Java OOP (provisional)

The exact official CSC2104 July-November 2026 course handout has not yet been supplied. The map below follows the Java assignment material currently available and the draft constraints; verify and adjust it against the official handout before declaring full course compliance.

| Concept | Implementation location |
|---|---|
| Classes, objects, encapsulation | Passenger, Booking, JourneyClass, SeatInventory, DisruptionCase, CaseEvent, RefundRule. |
| Constructors and validation | Validate PNR, fare, class and event construction; reject invalid states at boundaries. |
| Inheritance / abstract classes / overriding | Abstract SeatClass with Sleeper, AC and General subclasses; abstract RefundPolicy with LateRunning and TrainCancelled policy subclasses. |
| Interfaces / polymorphism | BookingBackend and RefundPolicy/RuleEvaluator interfaces; console works through contracts rather than concrete implementation. |

| Concept | Implementation location |
|---|---|
| Exception handling | Illegal transitions, duplicate confirmed booking, missing rule, unsupported case, and invalid input. |
| Arrays / references / data structures | Fixed-size seat inventories and fixed-array FIFO queue if confirmed by handout; separate per-run fixtures. |
| Thread / Runnable / synchronization | At least three agents, bounded queue, synchronized seat check-and-claim/release, staged unsafe/safe race, shutdown and join. |
| File I/O | Synchronized append-only audit log, fixture loading and deterministic summary export. |
| Object copy / inner classes / other explicit topics | Use only where required by the official handout; unrelated topics should be demonstrated in a small separate exercise rather than distorting the domain model. |

### 10.2 CSC2131 RDBMS Lab

| Lab / topic | Concrete project work |
|---|---|
| ER modelling, keys, normalization | ER diagram for 16 tables; candidate keys and 3NF justification; distinguish current allocation from booking/audit history. |
| DDL and data types | Create InnoDB tables, indexes, DATETIME, DECIMAL and constrained status/code fields. |
| Integrity constraints | PK/FK/UNIQUE/NOT NULL/CHECK; enforce unique PNR, one current seat per booking, one booking per seat and one waitlist entry per booking. |
| DML and SELECT / WHERE | Seed, book, cancel, evaluate; filter by status, class, rule and date. |
| Functions, aggregates, GROUP BY, HAVING | Occupancy by class; confirmed/waitlisted totals; average fare/refund summaries. |
| Joins | Inner/outer/self/cross/NATURAL JOIN demonstrations; explain why NATURAL JOIN is brittle and use explicit joins for application queries. |
| Subqueries | Single-row, multi-row and correlated subqueries for duplicate allocations, confirmed status and waitlist eligibility. |
| Views | Curated waitlist and latest-evaluation views; explicitly test the updatable view required by the lab. |
| Privileges, GRANT / REVOKE (Lab 11) | Separate least-privilege engine writer, read-only report reader and administrator; SHOW GRANTS after grant/revoke. |
| TCL, COMMIT / ROLLBACK / SAVEPOINT (Lab 12) | Booking/cancel transactions; group reservation savepoint per member; prove rolled-back writes and audit rows do not persist. |
| Triggers / procedures / cursor (Lab 12) | Audit insert/status changes, enforce status transitions, booking/cancel/promote/evaluate procedures, cursor-based waitlist promotion and/or summary processing. |

## 11. Implementation phases and scope control

The earlier target mentioned a 10 November 2026 submission. If that deadline remains current, the remaining time from this revision date is short. The team must confirm the actual deadline and adjust the plan immediately. Do not use the optional Tier 3 work as a reason to delay either academic core.

| Phase | Work | Exit gate |
|---|---|---|
| 0. Freeze (Day 1) | Obtain official CSC2104 handout; confirm MySQL acceptance/version; review rule sources; freeze fixtures and ER/class/state diagrams. | No unresolved course restriction; mandatory seed rules have documented source/scope. |
| 1. Java model | Domain classes, lifecycle, sequential service, audit logger and refund policy model. | Booking/cancel sequential tests plus D-01 to D-14 expected outcomes pass in Java fixtures. |
| 2. Java concurrency | Fixed-array queue as allowed, 3+ agents, shared journey monitor, shutdown, unsafe/safe race. | R-01, R-02, R-04, R-05, R-07 pass. |
| 3. MySQL schema + queries | Reset/tables/seeds, constraints, ERD/3NF, query/view/index pack and GRANT/REVOKE. | Reset script reruns cleanly; query outputs and constraints verified. |
| 4. MySQL transactions | Booking/cancel/promote, lock-first journey row, class lock, triggers/audit, occupancy function, savepoint demo and two-session experiments. | R-03 to R-10 pass; transaction failures leave no partial committed state. |
| 5. Refund advisor + parity | Rule-family/version tables, case events/evaluations/code links, procedures, shared fixtures and canonical parity runner. | D-01 to D-14 pass in both engines; parity matches exact canonical fields. |
| 6. Evidence and viva | Capture genuine outputs, update README, prepare ER/class/state diagrams, test checklist and mock viva. | Fresh-checkout run reproducible; each member explains the full project and defends their slice. |
| Future only | JDBC/HTTP/frontend; linked connection only after separate official scope review. | All previous gates pass and the team has explicit approval/time for extension. |

## 11.1 Cut order

If time is short, cut in this order: Tier 3 first; partial-cancellation and other refund cases (already out of MVP); any optional report polish; then optional query niceties not explicitly graded. Do not cut safe/unsafe concurrency evidence, transaction correctness, RDBMS Labs 11/12 content required by the rubric, the two mandatory refund rule families, or Java/MySQL parity on D-01 to D-11. If those mandatory gates cannot fit the confirmed deadline, reduce refund-advisor features before weakening the reservation core.

## 11.2 Team ownership (proposed)

| Owner area | Work products | Viva depth |
|---|---|---|
| Teammate with inheritance-level Java | Passenger/Booking/SeatClass hierarchy, class fare overrides, custom exceptions, file audit log; seed data, reporting SELECTs/views and privileges. | Explain classes, inheritance, overriding, state validation, keys, joins and views. |
| Project lead / strongest technical member | Journey-level monitor and queue, unsafe/safe race, MySQL lock-first procedures, transactions, savepoints, refund evaluator and parity. | Explain race, locking order, rollback semantics, rule precedence and Java/MySQL parity. |
| Other teammate(s) | Plain-language fixtures, official rule source records, SQL execution evidence, README, report and demo walkthrough; optionally simpler model classes and queries. | Explain user scenarios, expected outcomes and how to read the SQL/test evidence. |

*Every member must explain the project in 30 seconds and defend their own work in depth. Each person commits under their own name and must understand the integration points they modify.*

# 12. Risks and decisions required before freeze

| Risk / open decision | Required action | Status |
|---|---|---|
| Official CSC2104 restrictions | Obtain official handout and confirm whether fixed arrays, no Collections, monitor synchronization, interfaces, priority/join and specified file I/O are required/permitted. | BLOCKER for final Java queue design |
| MySQL accepted for assessed lab? | Confirm MySQL 8 is allowed for the project portion even if the lab handout names Oracle Academy/PLSQL topics. Document MySQL equivalents. | Confirm with faculty |
| Refund-rule source review | Record exact clause/source URL, title, checked date and a second review for both supported rule families before setting verified=true. | Required before final rule seeding |
| Linked connection | Removed from MVP. Current official IRCTC train-specific page says connecting train tickets are not applicable on IRCTC trains and no refund applies on connecting journey tickets. Do not restore without a separate, relevant official source and ticket scope. | Closed for MVP |
| Cross-class double confirmation | Journey-level lock/monitor is the common serialization point before class-level lock; one consistent order for booking/cancel/promotion. | Critical design fix applied |
| Deadline evaluation | DEADLINE_EXPIRED requires explicit filing attempt at/after actual departure; evaluation time alone cannot prove it. | Critical design fix applied |
| Rule source unverified | Unverified rule yields UNRESOLVED, not ELIGIBLE/NOT_ELIGIBLE. | Critical design fix applied |
| Scope and submission date | Confirm actual date; do not plan against a 4-5 week schedule if less time remains. | Confirm with team/faculty |
| Concurrent rule editing | Rule publication uses a rule-family lock and rejects overlapping effective ranges. Existing evaluations keep their chosen rule/event snapshots. | Required |

## 12.1 Freeze checklist

- Confirm course handout and database-dialect requirements.
- Agree the revised lock order and invariants INV-01 to INV-07.
- Agree the two supported refund rule families and mark seed rules unverified until reviewed.
- Approve fixture R-01 to R-10 and D-01 to D-14 and the exact parity fields.
- Confirm project owners and a date-based schedule.
- Commit this SRS version as the implementation baseline; later changes require a versioned decision entry.

# 13. References

[1] Indian Railway Catering and Tourism Corporation (IRCTC), “Refund Rules and TDR Filing,” official PDF: https://contents.irctc.co.in/en/RefundCancellationRules.pdf (accessed 24 October 2026). Relevant clauses include 5(a), cancellation of e-tickets for cancelled trains, and 6, train running more than three hours late and passenger not travelled. The document contains rule text with later amendments; the precise rule scope must be checked before a rule is marked verified.

[2] IRCTC, “Cancellation and Refund Rules for IRCTC Trains,” official PDF: https://contents.irctc.co.in/en/CancellationRulesforIRCTCTrain.pdf (accessed 24 October 2026). Relevant sections include the late-running >3-hour case and a specific note that connecting-train ticket booking is not allowed on IRCTC trains and no refund is applicable on connecting-journey tickets.

[3] IRCTC, “E-Ticket Cancel,” official guidance: https://contents.irctc.co.in/en/eticketCancel.html (accessed 24 October 2026). Used as a supplementary source for current cancellation/TDR workflow context.

[4] CSC2131 RDBMS Lab Course Handout, provided by the project team. Use the official copy for syllabus traceability, particularly Labs 11 and 12: privileges, TCL, triggers, procedures and transactions.

[5] CSC2104 Java course handout, July-November 2026: not yet provided in the accessible project materials. Must be obtained and checked before finalizing the Java restriction matrix.

[6] Previous document: “Software Requirements Specification: Railway Reservation Simulator,” 21 October 2026, 14 pages. This revision restores its detailed reservation invariants, queue/agent requirements, query list and concurrency acceptance tests, with corrections described in revision history.

# Appendix A. Canonical fixture matrix

This matrix is the required test contract. Every fixture should be expressed in plain language first and then in JSON/CSV. Expected results are fixed test expectations, not claims about actual railway cases.

## A.1 Reservation fixtures

| ID | Setup/action | Expected result / invariant |
|---|---|---|
| R-01 | Unsafe Java race with two requests and one seat, controlled by a start gate. | Over-allocation occurs in isolated unsafe inventory; evidence proves the race. |
| R-02 | Safe Java race, 10 agents, 100 repeated runs. | Never exceeds capacity; one confirmed, remaining valid requests waitlisted or otherwise resolved by policy. |
| R-03 | Two MySQL sessions execute booking procedure for last seat. | Second waits at JOURNEY lock and cannot overbook. |
| R-04 | Cancel a confirmed allocation with two waitlisted candidates. | Exactly one earliest eligible candidate promoted. |
| R-05 | Earliest waiter has confirmed reservation in another class. | Skipped unchanged; next eligible waiter may be promoted. |
| R-06 | Cancel waitlisted booking; then cancellation with empty list. | Waitlisted cancel frees no seat; empty promotion list exits cleanly. |
| R-07 | Try each forbidden transition. | Rejected, no illegal state or partial audit. |
| R-08 | Three-member group reservation; member two fails after partial side effect. | Only member-two work since savepoint undone; other valid members commit once. |
| R-09 | Inject an SQL error after insert/update but before commit. | Entire operation rolls back, including audit rows. |
| R-10 | Run invariant query suite on valid seed and isolated unsafe demo. | Valid suite returns zero violations; unsafe demo is detected by intended checker. |

## A.2 Refund fixtures

| ID | Condition | Expected result |
|---|---|---|
| D-01 | Official simulated delay = 210 minutes; not travelled; filing strictly before actual departure. | ELIGIBLE; full simulated fare; delay and before-departure codes. |
| D-02 | Delay exactly 180 minutes. | NOT_ELIGIBLE / DELAY_NOT_OVER_THRESHOLD. |
| D-03 | Delay 179 minutes. | NOT_ELIGIBLE / DELAY_NOT_OVER_THRESHOLD. |

| ID | Condition | Expected result |
|---|---|---|
| D-04 | Passenger-only delay claim; no official-simulated delay observation. | UNRESOLVED / DELAY_UNVERIFIED. |
| D-05 | Missing delay or travel-state fact. | UNRESOLVED / specific missing-fact code. |
| D-06a | Filing at actual departure minus one second. | ELIGIBLE if all other criteria pass. |
| D-06b | Filing exactly at actual departure. | DEADLINE_EXPIRED / FILED_AFTER_DEPARTURE. |
| D-06c | Filing after actual departure. | DEADLINE_EXPIRED / FILED_AFTER_DEPARTURE. |
| D-06d | No filing-attempt event. | UNRESOLVED / FILING_ATTEMPT_UNKNOWN. |
| D-07 | Passenger travelled. | NOT_ELIGIBLE / PASSENGER_TRAVELLED. |
| D-08 | Official-simulated train cancellation on confirmed simulated e-ticket. | ELIGIBLE; full modelled fare; NO_TDR_REQUIRED. |
| D-09 | Two rule versions; applicability time selects older version while evaluation time is later. | The interval-applicable version is used. |
| D-10 | No rule effective at applicability time. | UNRESOLVED / RULE_NOT_FOUND. |
| D-11 | Selected rule has verified=false. | UNRESOLVED / RULE_SOURCE_UNVERIFIED. |
| D-12 | Waitlisted/unsupported ticket scope. | Reject case evaluation with UNSUPPORTED_TICKET_SCOPE. |
| D-13 | Java/MySQL same fixed fixture. | All parity fields equal. |
| D-14 | Conflicting essential event records. | UNRESOLVED / EVENT_CONFLICT. |

# Appendix B. Example deterministic refund fixture

Illustrative JSON shape; use the exact field names agreed by both implementations. This example uses synthetic values only.


```
{
  "fixtureId": "D-01-late-more-than-180-minutes-v1",
  "ruleFamily": "LATE_OVER_3_HOURS_E_TICKET",
  "ruleVersion": 1,
  "verifiedRuleFixture": true,
  "ticketScope": "SIMULATED_E_TICKET_CONFIRMED",
  "policyApplicabilityTime": "2026-11-03T08:00:00+05:30",
  "evaluationTime": "2026-11-03T14:25:00+05:30",
  "fareINR": "1250.00",
  "events": [
    {"type":"SCHEDULED_DEPARTURE", "eventTime":"2026-11-03T08:00:00+05:30", "source":"SIMULATED_OFFICIAL"},
    {"type":"DELAY_AT_BOARDING_MINUTES", "value":210, "eventTime":"2026-11-03T14:00:00+05:30",
"source":"SIMULATED_OFFICIAL"},
    {"type":"PASSENGER_NOT_TRAVELLED", "eventTime":"2026-11-03T14:10:00+05:30", "source":"PASSENGER_REPORTED"},
    {"type":"FILING_ATTEMPT", "eventTime":"2026-11-03T14:20:00+05:30", "source":"SIMULATED_INPUT"},
    {"type":"ACTUAL_DEPARTURE", "eventTime":"2026-11-03T14:35:00+05:30", "source":"SIMULATED_OFFICIAL"}
  ],
  "expected": {
    "outcome":"ELIGIBLE",
    "reasonCodes":["DELAY_OVER_THRESHOLD", "FILED_BEFORE_DEPARTURE"],
    "warnings":["PASSENGER_STATUS_SELF_REPORTED"],
    "deadlineAt":"2026-11-03T14:35:00+05:30",
    "actionCode":"MODELLED_TDR_FILED_BEFORE_DEPARTURE",
    "currency":"INR",
```

```
  "amount":"1250.00"
 }
}
```

The example is not an official refund result. Project fixtures may label modeled facts as SIMULATED_OFFICIAL only to test the evaluator; the disclaimer must remain in every generated case summary.

# Appendix C. RDBMS query and lab-topic checklist

The following query demonstrations should be captured with representative output in the implementation repository. They are syllabus evidence, not optional product features when required by the course rubric.

| Topic | Required demonstration |
|---|---|
| SELECT / WHERE / ORDER BY / LIMIT | Find confirmed bookings, bookings by class/date, and top/limited recent booking records. |
| Comparison/logical/string operators | Fare ranges, class filters, status combinations, LIKE patterns and NULL checks where applicable. |
| Aggregates / GROUP BY / HAVING | Occupancy, confirmed/waitlisted counts, and groups/classes over a chosen count threshold. |
| Joins | Inner join booking-passenger-journey; outer join waitlisted bookings with optional allocations; self-join rule version history; cross join class template; natural join solely as a teaching example with warning. |
| Subqueries | Scalar current fare/policy threshold, IN list of occupied classes, correlated query for passenger journey booking conflicts. |
| Functions / COALESCE | occupancy_pct and handling zero seats or no matching evaluations. |
| Views | Waitlist ordered view, latest evaluation per case view and one appropriately updatable view. |
| GRANT / REVOKE / SHOW GRANTS | Read-only report user versus procedure execution/writer user; verify grants and revocation. |
| TCL | Explicit COMMIT, ROLLBACK and SAVEPOINT; show the result after rollback and after partial rollback. |
| Procedure / trigger / cursor | Book/cancel/promote/evaluate procedures; lifecycle/audit triggers; cursor that iterates waitlist or refreshes a report. |
| MERGE / multi-table insert dialect note | Explain that MySQL differs from Oracle PL/SQL syntax; do not paste Oracle-only MERGE/INSERT ALL into MySQL scripts. |